**Universidad de Ibagué**
**Department of Logistics and Computer Sciences**
**Systems Engineering Programme**
**Web, APIs and Microservices**

# PROYECTO 03

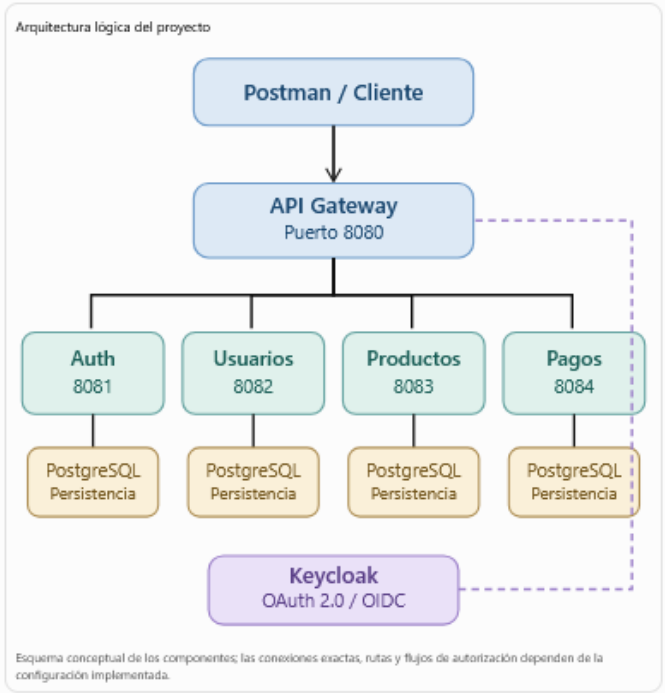
## Spring Boot 3.0 con microservicios, Docker, Kubernetes, PostgreSQL y Keycloak

El proyecto contiene cinco servicios Spring Boot:

| Componente | Puerto |
|---|---|
| API Gateway | 8080 |
| Autenticación | 8081 |
| Usuarios | 8082 |
| Productos | 8083 |
| Pagos | 8084 |
| Keycloak (acceso local) | 8085 |

Incluye cuatro bases de datos PostgreSQL, autenticación JWT, OAuth 2.0, manifiestos de Kubernetes y un flujo de integración continua para GitHub Actions.

### Arquitectura lógica del proyecto



Se Verifica que Docker Desktop tiene Kubernetes habilitado. En Git Bash, se ejecuta el comando:

**kubectl config use-context docker-desktop** y **kubectl get nodes** **(Figura 1).**

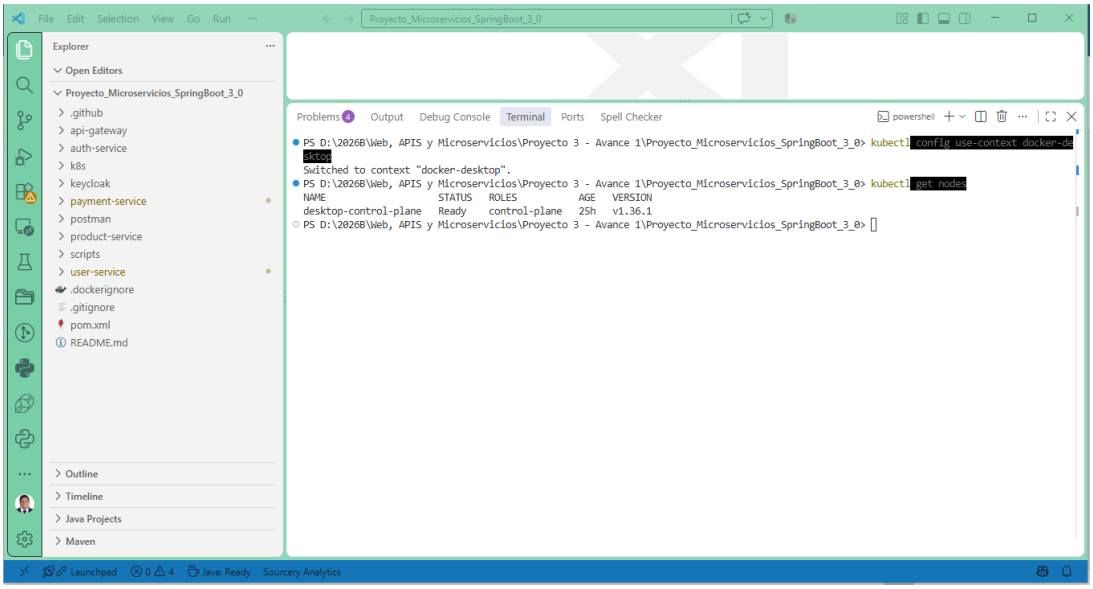



**Figura 1.**

Se genera las claves de seguridad RSA, con el comando: **bash scripts/generate-rsa-keys.sh** **(Figura 2).**

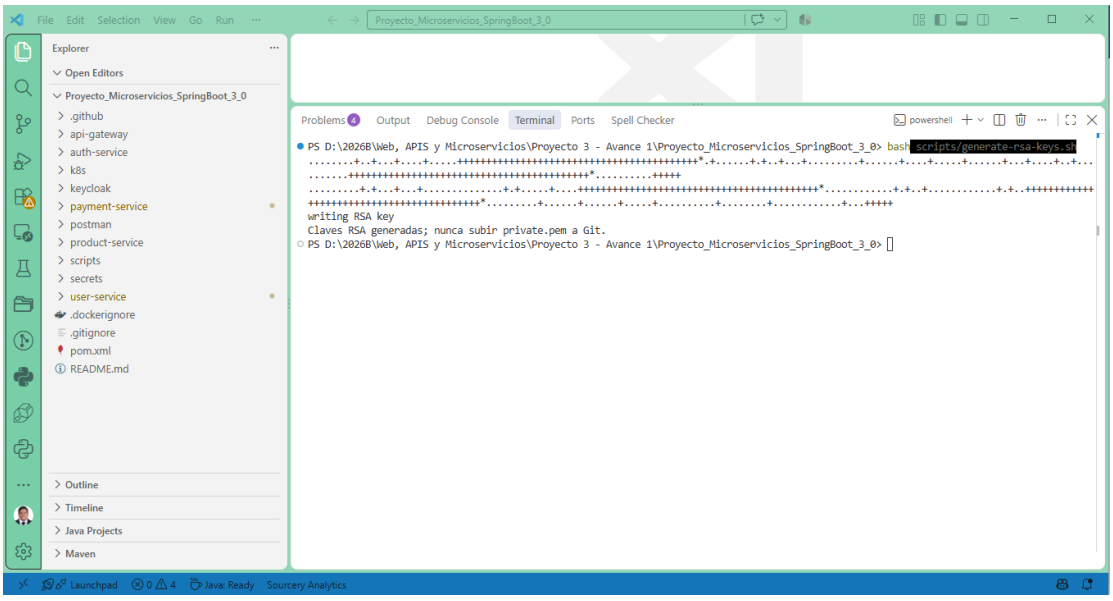



**Figura 2.**

Se construye las imágenes Docker, con el comando: **bash scripts/build-images.sh (Figuras 3, 4, 5, 6 y 7).**

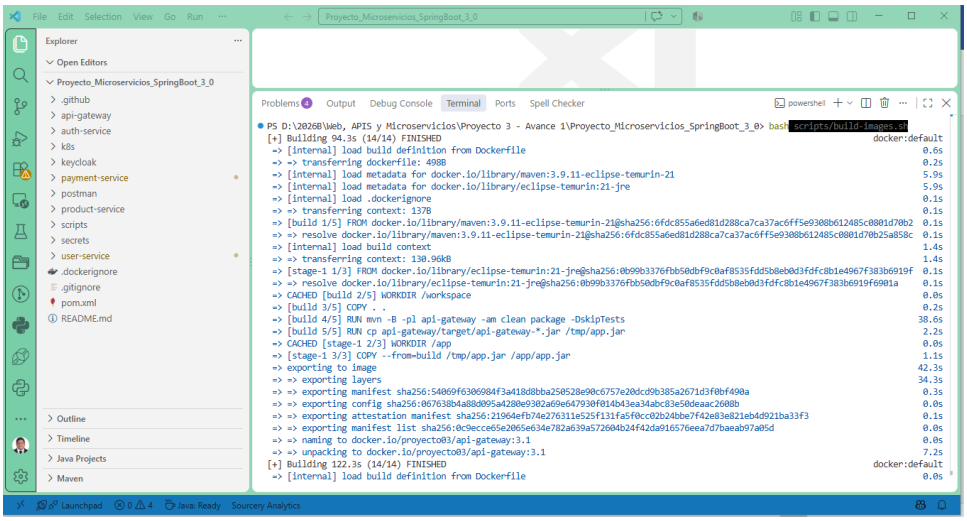



**Figura 3.**

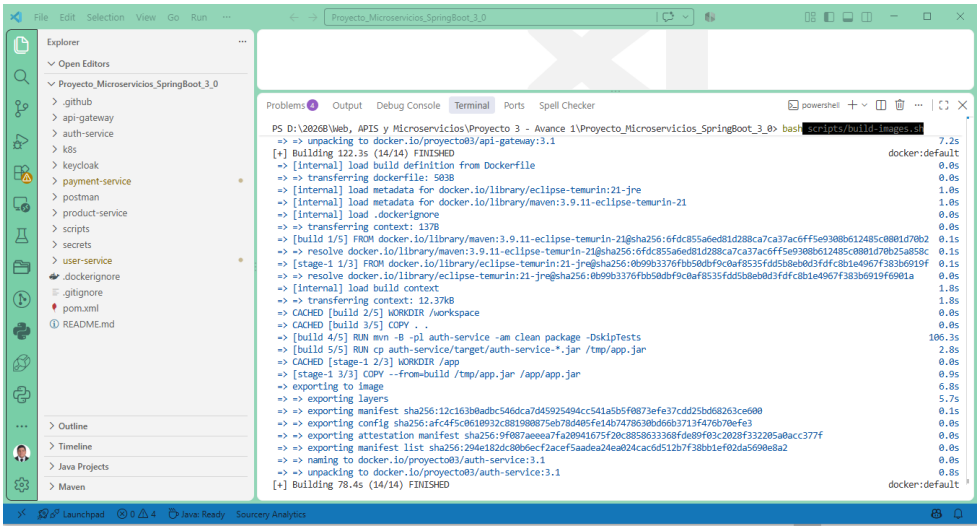



**Figura 4.**

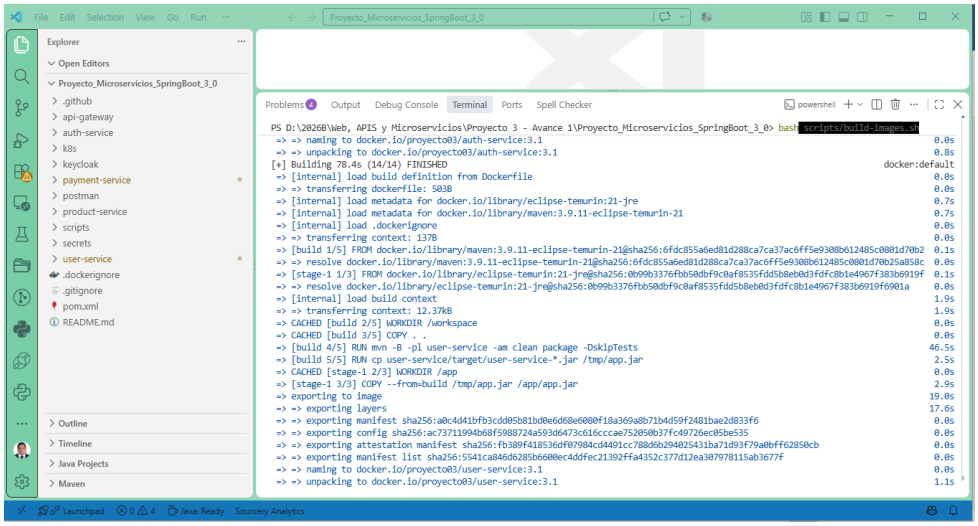



**Figura 5.**

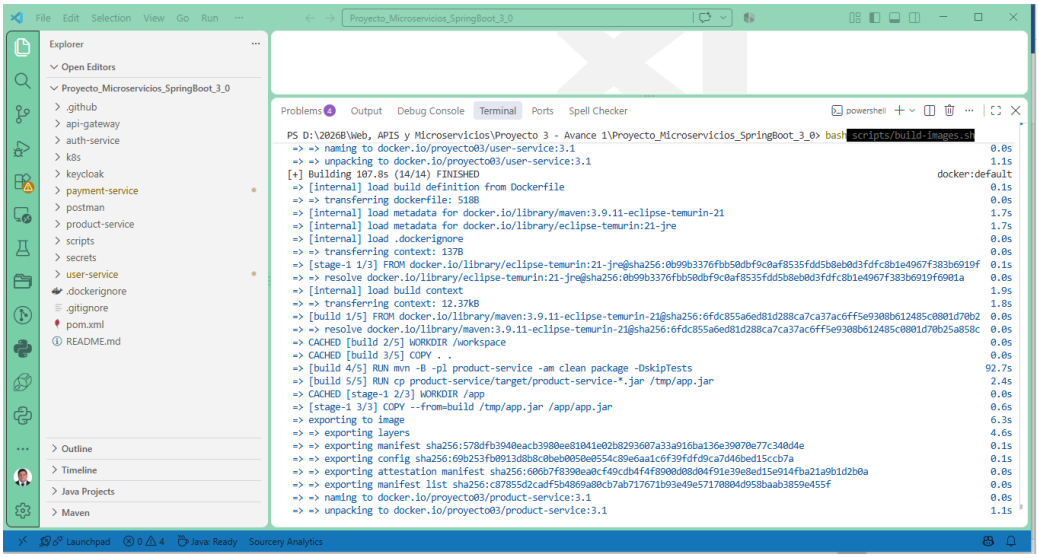



**Figura 6.**

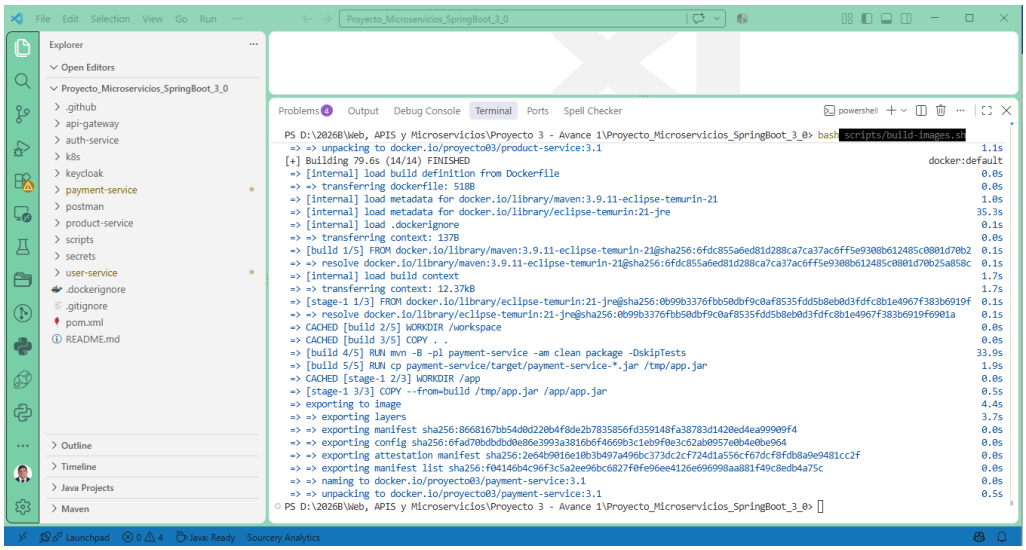



**Figura 7.**

Se generarán las cinco contraseñas criptográficamente seguras y diferentes para el proyecto Spring Boot 3.0, utilizando OpenSSL en su propio computador. Así las credenciales no tendrán que compartirse. Se requieren cuatro contraseñas y un secreto HMAC. El secreto OAuth de Keycloak se genera automáticamente.

### Credenciales del Proyecto

| Servicio | Longitud generada |
|---|---|
| **PostgreSQL: autenticación y usuarios.** | **48 caracteres** |
| **PostgreSQL: productos.** | **48 caracteres** |
| **PostgreSQL: pagos.** | **48 caracteres** |
| **Keycloak: administrador.** | **48 caracteres** |
| **Webhook: secreto HMAC.** | **48 caracteres** |

Todas serán cadenas hexadecimales aleatorias, generadas mediante un generador criptográficamente seguro.

**Generar las contraseñas**

Se abre Git Bash en Windows y se ejecuta los siguientes comandos:

**echo "1. PostgreSQL AUTH y USERS:"**
**openssl rand -hex 24**

**echo "2. PostgreSQL PRODUCTS:"**
**openssl rand -hex 24**

**echo "3. PostgreSQL PAYMENTS:"**
**openssl rand -hex 24**

**echo "4. WEBHOOK HMAC:"**
**openssl rand -hex 32**

**echo "5. KEYCLOAK ADMIN:"**
**openssl rand -hex 24**

Se obtiene los siguientes resultados:

**Unibague@U15280501481 MINGW64 /d/2026B/Web, APIS y Microservicios/Proyecto 3 - Avance 1/Proyecto_Microservicios_SpringBoot_3_0**
**$ echo "1. PostgreSQL AUTH y USERS:"**
**openssl rand -hex 24**

**echo "2. PostgreSQL PRODUCTS:"**
**openssl rand -hex 24**

**echo "3. PostgreSQL PAYMENTS:"**
**openssl rand -hex 24**

**echo "4. WEBHOOK HMAC:"**
**openssl rand -hex 32**

**echo "5. KEYCLOAK ADMIN:"**
**openssl rand -hex 24**
**1. PostgreSQL AUTH y USERS:**
**c14d9910e5e7a0b826fecdf00b15130c5dc4ab336396b9e4**
**2. PostgreSQL PRODUCTS:**

**d053ee63b6322c48afb10b5189893f739f614c854f968e5d**
**3. PostgreSQL PAYMENTS:**
**8809d203f2f5c0cba1489417df03728e301b54b34f59060b**
**4. WEBHOOK HMAC:**
**30114c641dc886819052081c5e8f246ad7dd441064a65748bf8b73756512b47b**
**5. KEYCLOAK ADMIN:**
**b4f48ee56b93b5328bd2e99cb3587690365b9f0454ffb736**

**Unibague@U15280501481 MINGW64 /d/2026B/Web, APIS y Microservicios/Proyecto 3 - Avance 1/Proyecto_Microservicios_SpringBoot_3_0**

**$**

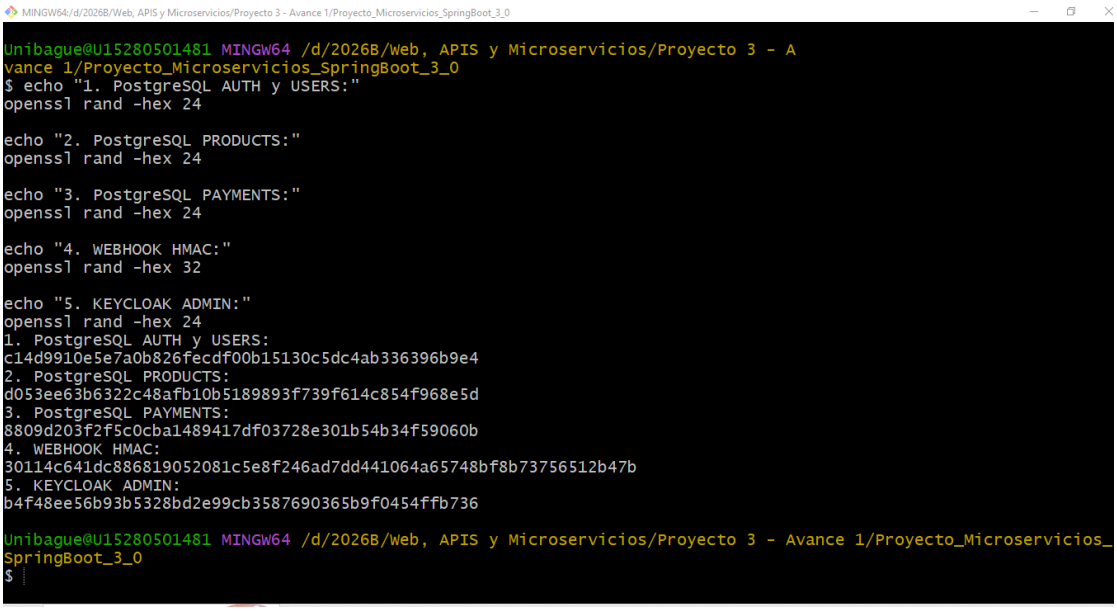



**Figura 8.**

**Nota importante:** Cada ejecución generará un valor nuevo, por lo que se debe guardar cada resultado en su gestor de contraseñas. No comparta aquí las contraseñas generadas.

Se despliega toda la aplicación con el comando: **bash scripts/k8s-setup.sh (Figuras 9, 10 y 11).**

Introduce cada contraseña en el orden indicado. Aunque escriba o pegue los valores, la terminal no mostrará los caracteres porque el script utiliza entrada oculta.

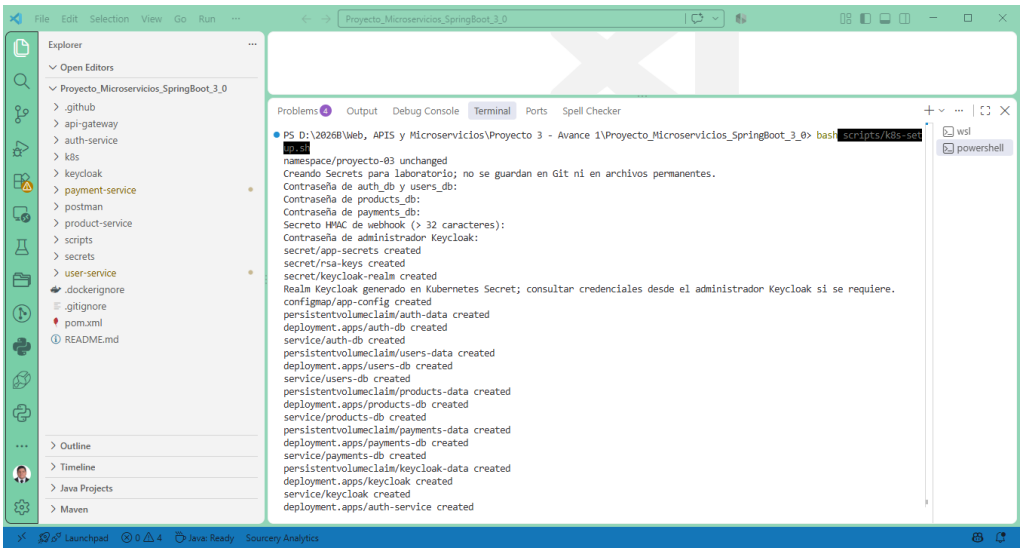



**Figura 9.**

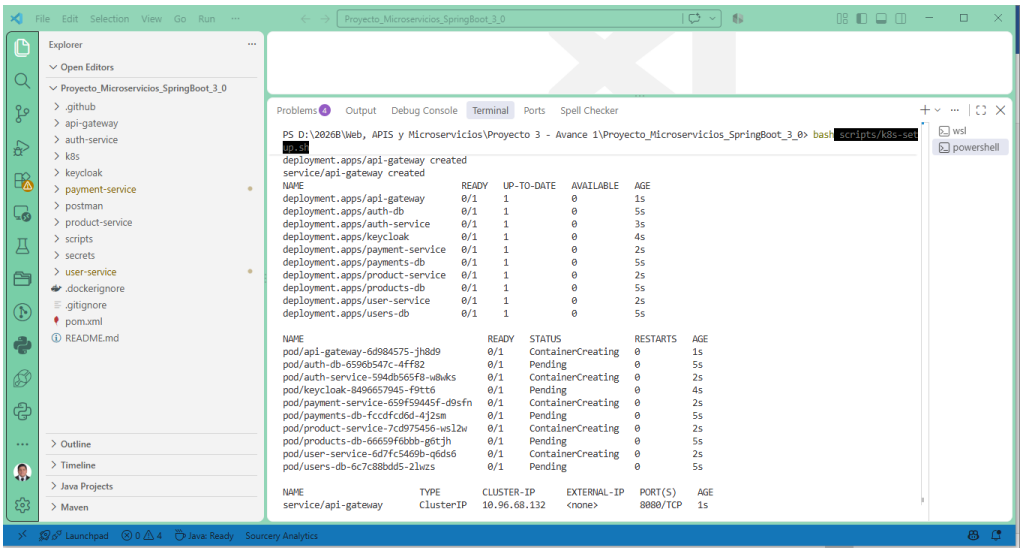



**Figura 10.**

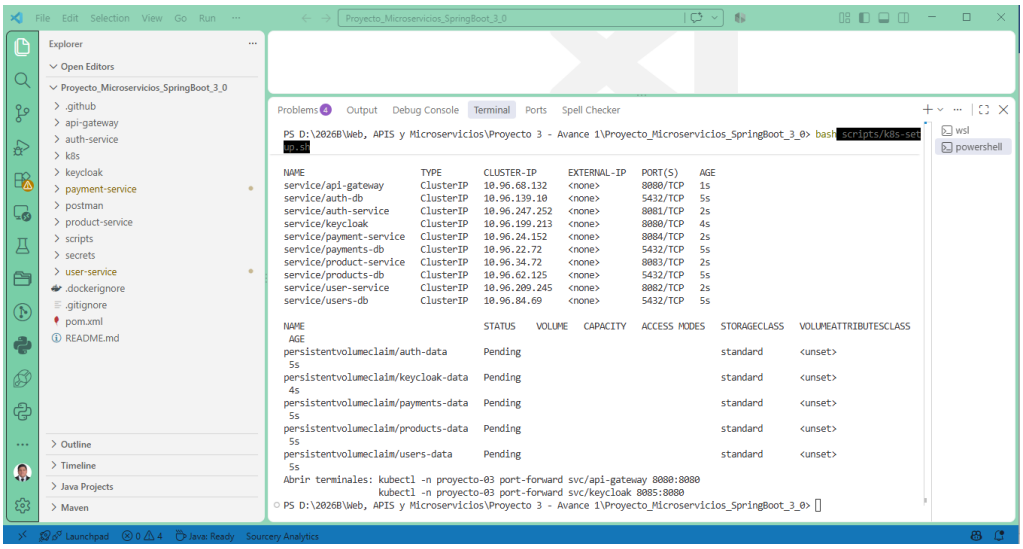



**Figura 11.**

Se verifica las credenciales nuevas, después de la instalación con los siguientes comandos:

**kubectl -n proyecto-03 get secret app-secrets**
**kubectl -n proyecto-03 get secret keycloak-realm**
**kubectl -n proyecto-03 get pods (Figura 12)**

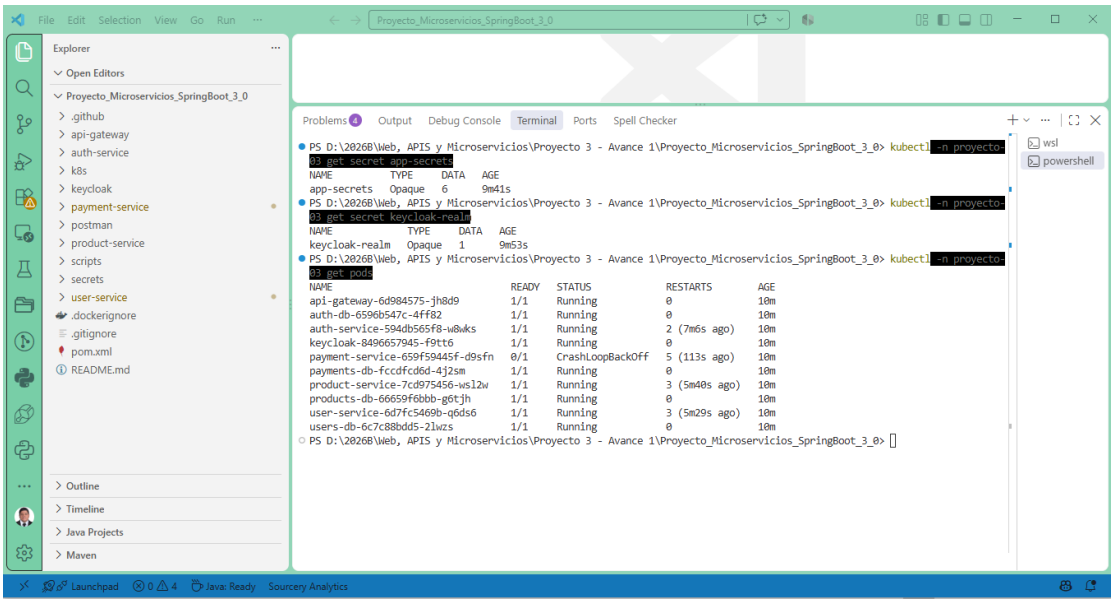



**Figura 12.**

Se verifica la nueva instalación cuando termine el script y se ejecutan los comandos:

**kubectl -n proyecto-03 get secret**
**kubectl -n proyecto-03 get pods**
**kubectl -n proyecto-03 get pvc (Figuras 13 y 14).**

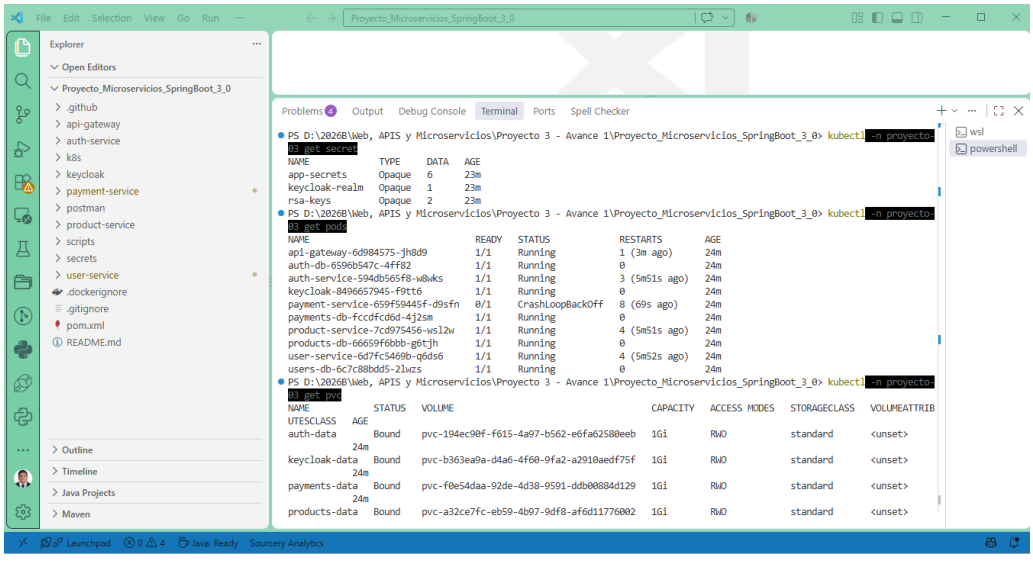



**Figura 13.**

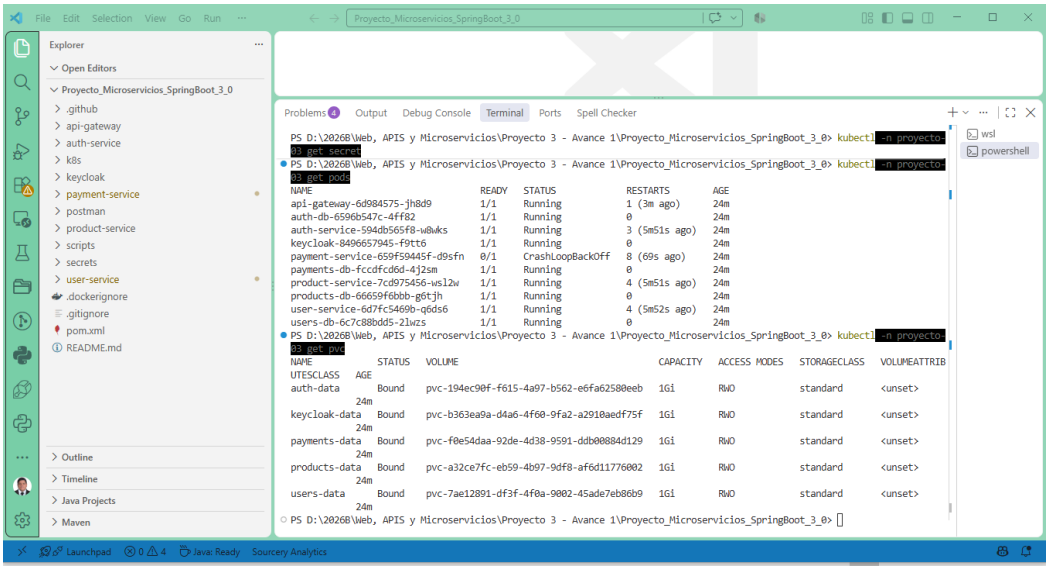



**Figura 14.**

Se encuentran los tres Secrets nuevamente creados, los Pods funcionando y los cinco PVC vinculados.

Se accede al API Gateway ejecutando el comando:

**kubectl -n proyecto-03 port-forward svc/api-gateway 8080:8080 (Figura 15).**

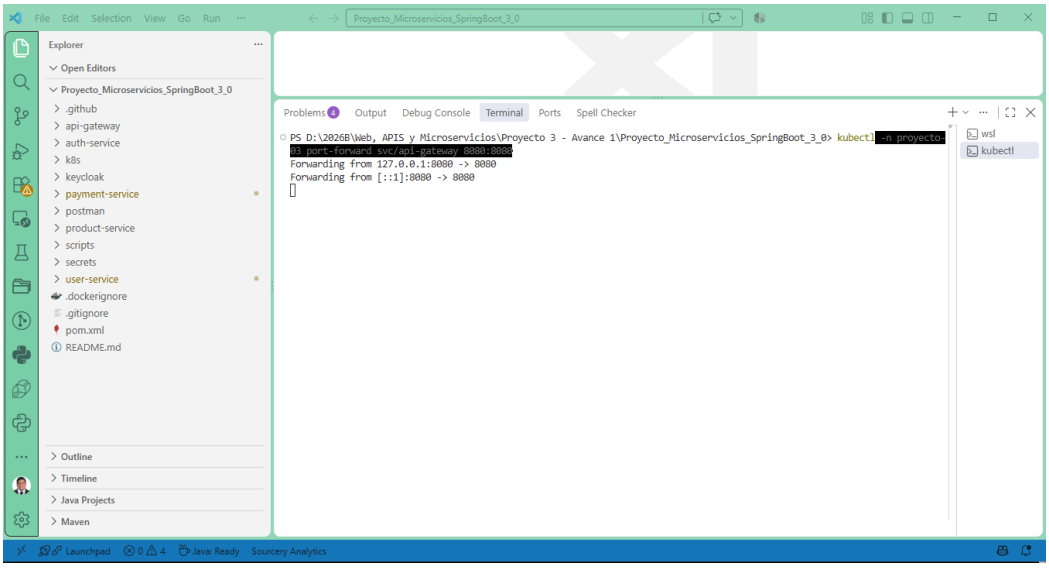



**Figura 15.**

Y consultar en un navegador web: http://localhost:8080/actuator/health. **(Figura 16).**

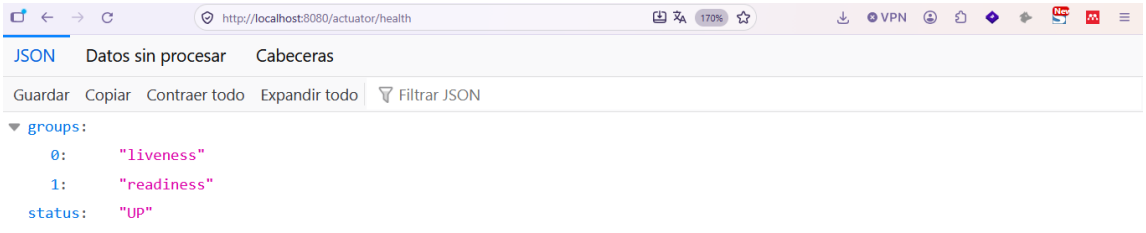



**Figura 16.**

Para publicar el proyecto en GitHub, se debe verificar que Git esté instalado en el equipo. Se abre la terminal integrada de Visual Studio Code: **Terminal → New Terminal**, y se ejecuta el comando: **git --version** **(Figura 17).**

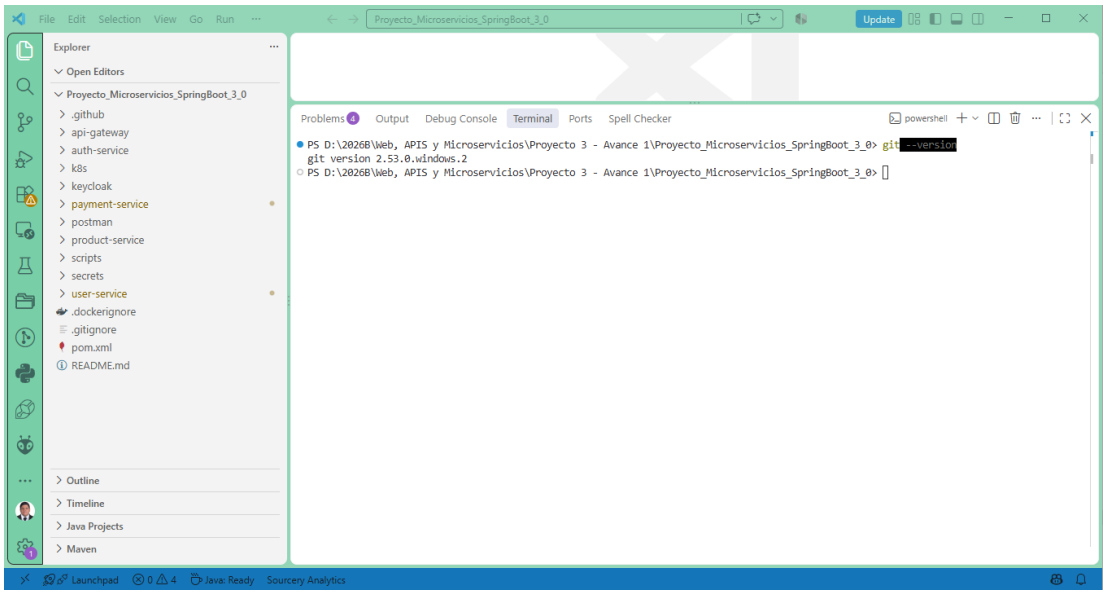



**Figura 17.**

Para la protección de contraseñas y archivos privados, el proyecto contiene credenciales de PostgreSQL, Keycloak, secretos HMAC y claves RSA. Antes de publicar, crea o revisa el archivo **.gitignore** en la raíz del proyecto:

```
# Archivos compilados
target/
*.class
*.jar
*.war

# Visual Studio Code
.vscode/
```

**# Secretos y claves privadas**
**.env**
**.env.***
**!.env.example**
**secrets/**
***.pem**
***.key**
***.p12**
***.jks**

**# Respaldos**
**respaldo-proyecto03/**
**respaldos-postgresql/**
***.dump**
***.sql**

**# Archivos temporales**
***.log**
**.DS_Store** **(Figura 18).**

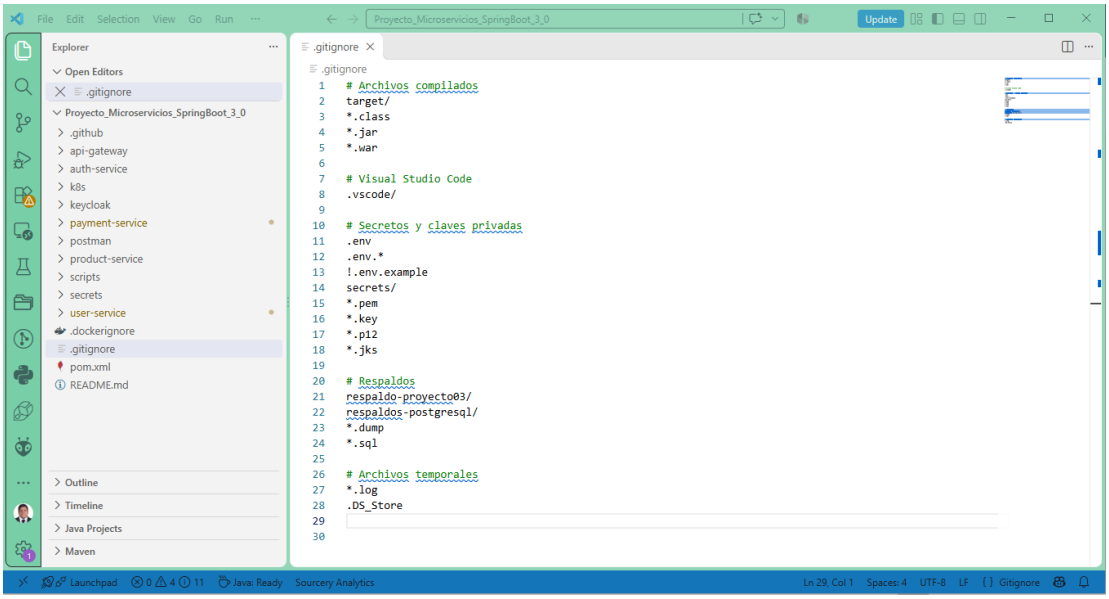



**Figura 18.**

Se recomienda no incluir contraseñas reales en archivos YAML, propiedades de Spring Boot, scripts ni documentos que se suban al repositorio. Si algún secreto ya estuvo versionado, añadirlo a **.gitignore** no se elimina del historial: habrá que retirarlo de Git y rotarlo.

Para publicar el proyecto desde Visual Studio Code:

1. Se da clic en el icono **Source Control (Control de código fuente)**, situado en la barra lateral izquierda, o presiona **Ctrl + Shift + G**. Si aparece **Initialize Repository**, haz clic para inicializar Git. **(Figura 19).**

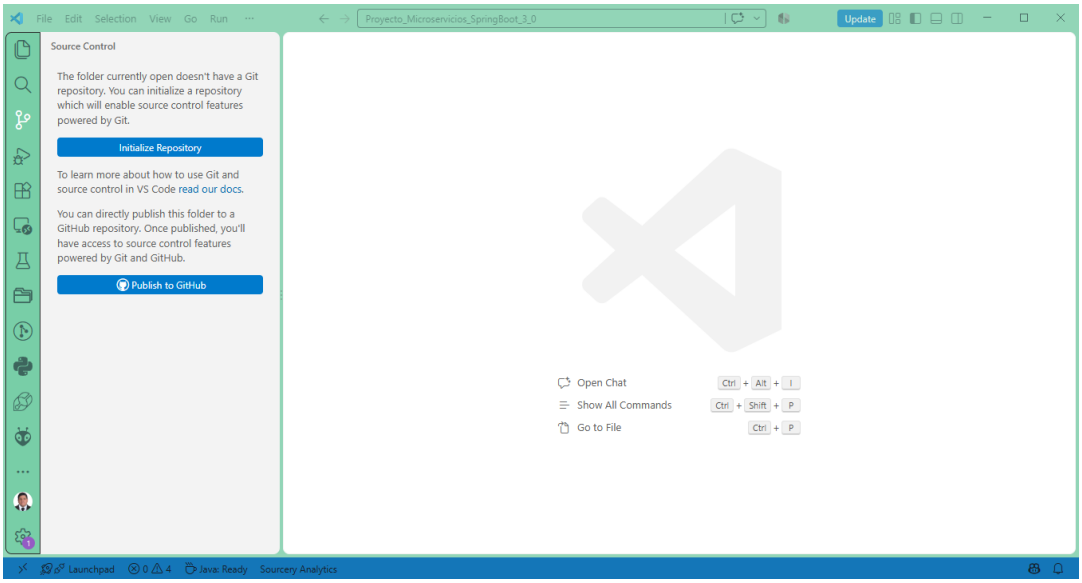



**Figura 19.**

Se revisa los archivos pendientes de publicación y se comprueba que no se incluyan secretos ni respaldos. Se escribe el mensaje de commit: Versión inicial - Microservicios Spring Boot 3.0. y se pulsa **Commit**. **(Figura 20).**

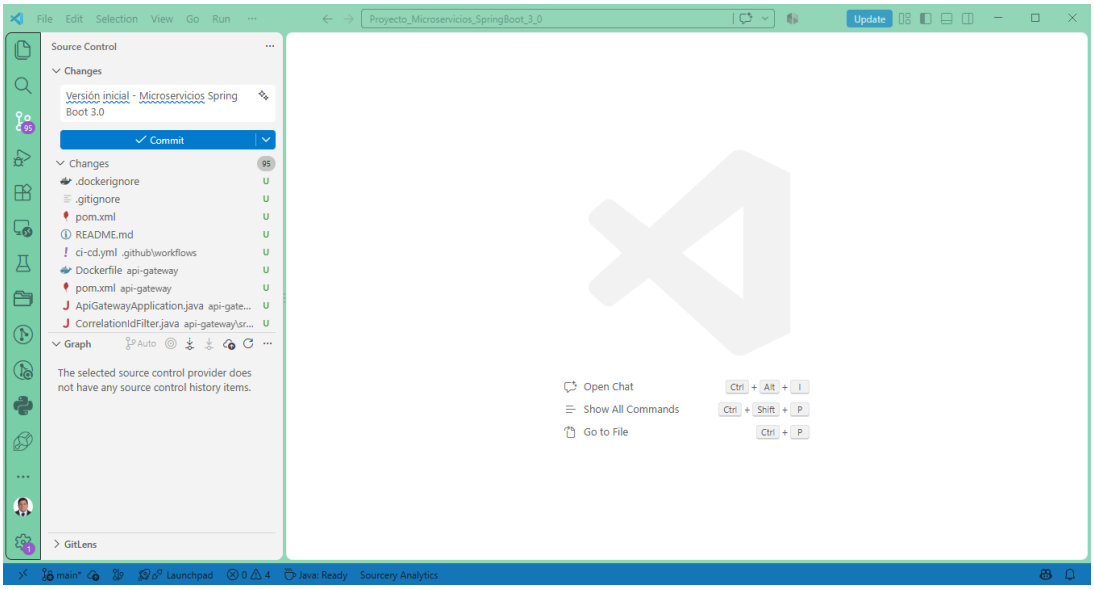



**Figura 20.**

Se da clic en la opción **Publish to GitHub**. **(Figura 21).**

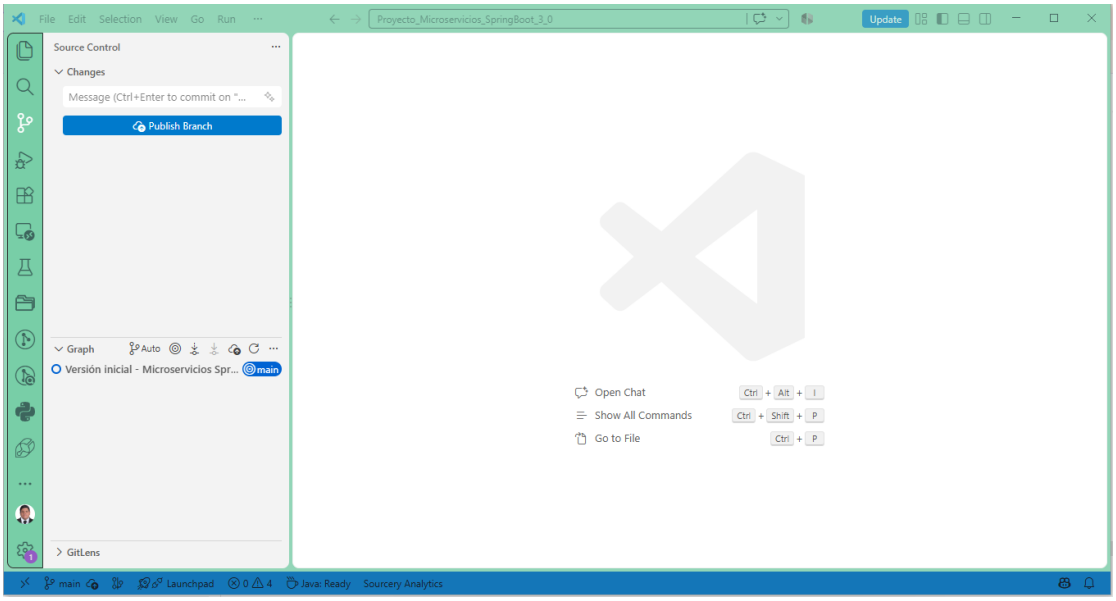



**Figura 21.**

Al iniciar sesión en GitHub, si VS Code lo solicita, se escribe el nombre del repositorio: **Proyecto-Microservicios-SpringBoot-3.0**. Se selecciona **Public Repository** inicialmente, mientras terminas de revisar la seguridad. Confirma la publicación. (**Figura 22).**

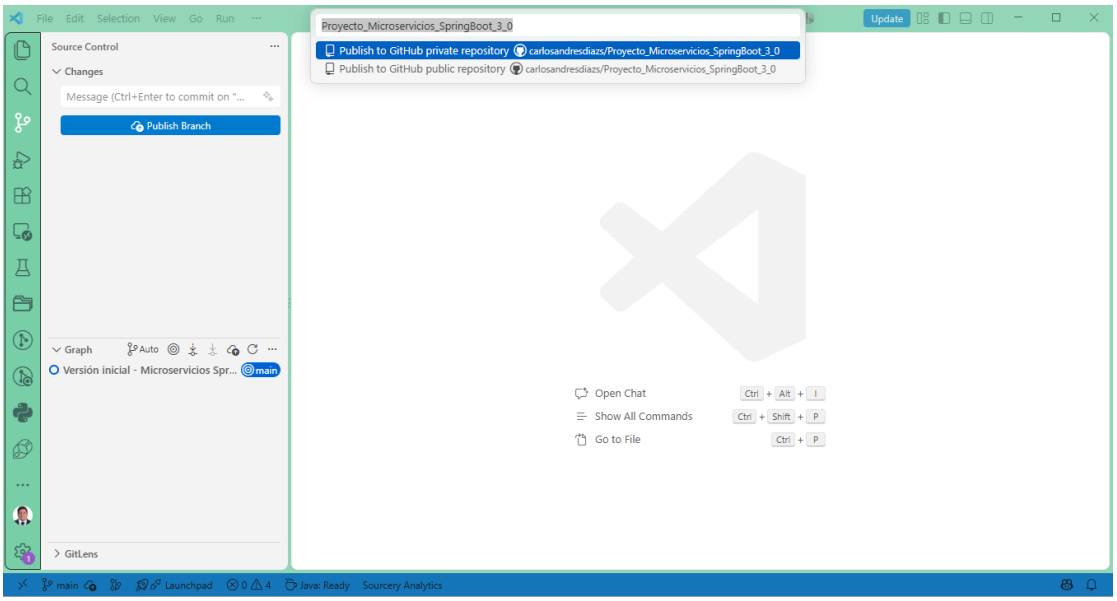



**Figura 22.**

Visual Studio Code creará el repositorio remoto y subirá los archivos. Si no aparece **Publish to GitHub**, presiona **Ctrl + Shift + P** y busca **Git: Publish to GitHub**.

Para verificar que se publicó correctamente, se abre el repositorio personal: https://github.com/carlosandresdiazs . Una vez publicado, se comprueba que aparezcan las carpetas de los microservicios, los scripts y los manifiestos de Kubernetes. **(Figura 23).**

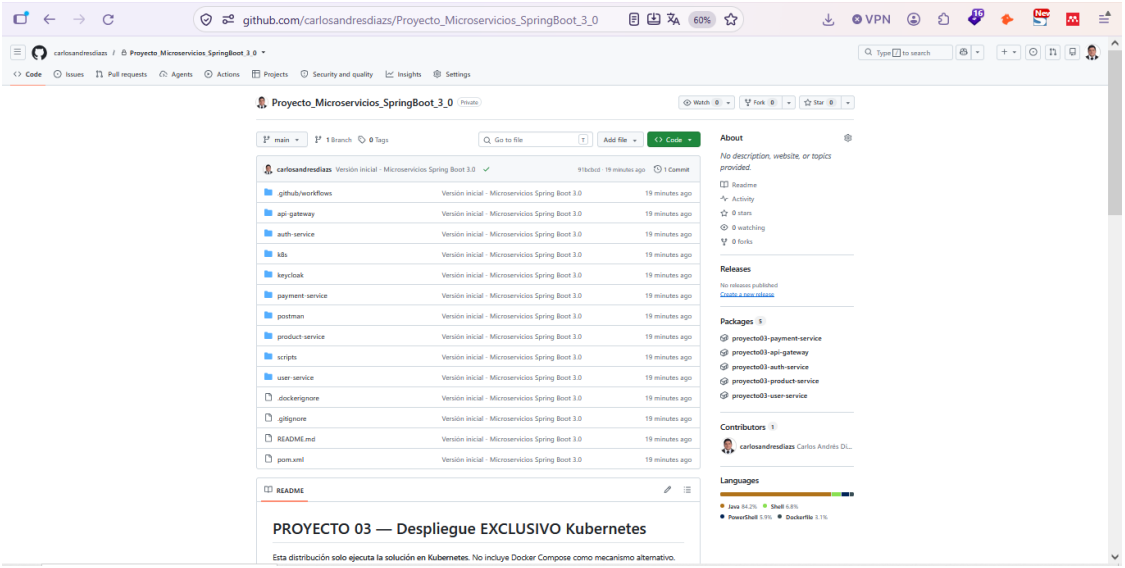



**Figura 23.**

También se puede verificar la conexión desde la terminal de VS Code, con los comandos: **git remote -v** y **git status. (Figura 24).**

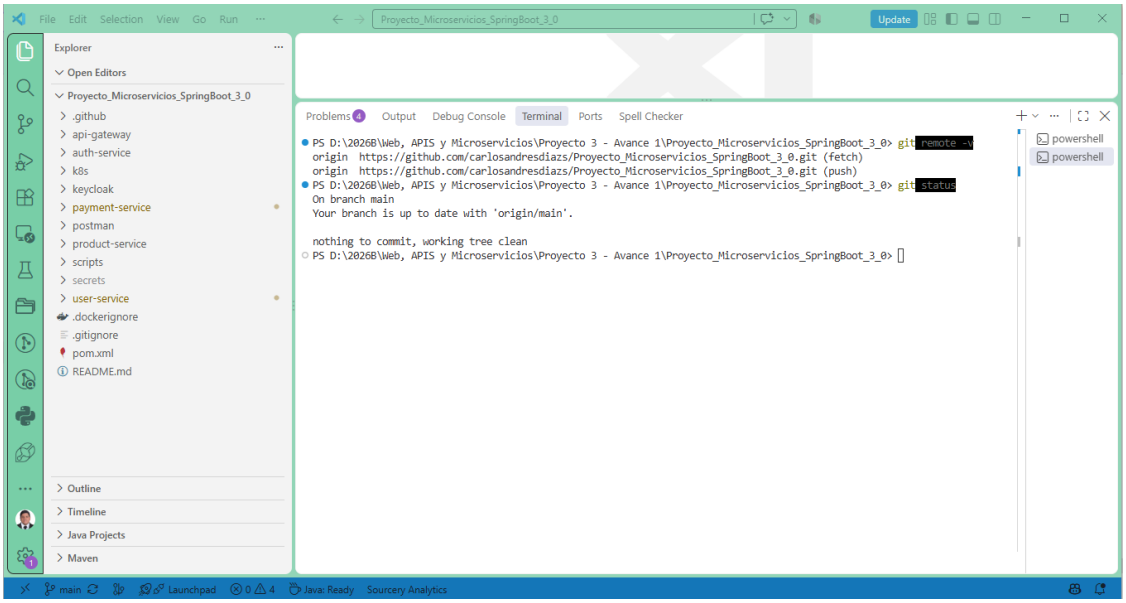



**Figura 24.**

El primer comando muestra una URL de GitHub. El segundo indica que el directorio de trabajo está limpio si no hay modificaciones pendientes.

**Alternativa: subir el proyecto mediante comandos Git**

Si prefiere controlar todo desde PowerShell, primero crea un repositorio vacío en **GitHub — Nuevo repositorio** sin inicializarlo con **README** ni **.gitignore**.

Después, en la carpeta principal del proyecto, se ejecutan los comandos:

**git init**

**git branch -M main**
**git add .**
**git status**

Se revisa cuidadosamente **git status**. Si los archivos preparados son correctos, se ejecutan los comandos:

**git commit -m "Version inicial Spring Boot 3.0"**

**git remote add origin https://github.com/carlosandresdiazs/Proyecto-Microservicios-SpringBoot-3.0.git**

**git push -u origin main**

Si origin ya existe, consulta el comando: **git remote -v** antes de cambiarlo. No repita el comando **git init** ni cree otro repositorio, si su proyecto ya está conectado correctamente a GitHub.

Para actualizar GitHub cuando se modifique el proyecto. Después de modificar controladores, servicios, Dockerfiles o archivos de Kubernetes, podrá subir los cambios desde VS Code con **Source Control** → **Commit** → **Sync Changes,** o desde la terminal, con los comandos:

**git add .**
**git commit -m "Actualización de microservicios"**
**git push**

**Conclusión:** Para su proyecto, lo más importante es publicar únicamente el código fuente y la configuración reproducible, sin claves RSA privadas, contraseñas de Kubernetes ni archivos de respaldo de PostgreSQL. Así podrá compartir el repositorio y reconstruir el entorno de forma segura.

Bibliografía:

ChatGPT OpenAI [06 de Noviembre de 2026]. Spring Boot 3.0 con microservicios, Docker, Kubernetes, PostgreSQL y Keycloak. https://chatgpt.com/share/6aee5e59-f6f4-83e9-8d7b-bab95cce576e